

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 3

Выполнил:

Берулава Леон Алхасович

К3439

Проверил:

Добряков Д. И.

Санкт-Петербург

2025 г.

Задача

- выделить самостоятельные модули в вашем приложении;
- провести разделение своего API на микросервисы (минимум, их должно быть 3);
- настроить сетевое взаимодействие между микросервисами.

Ход работы

В процессе анализа монолитного кулинарного API были выделены четыре основных функциональных модуля с четкими границами ответственности и минимальными зависимостями между собой. Модуль управления пользователями включает функциональность регистрации, аутентификации, управления профилями и хранения персональных данных пользователей с независимой схемой базы данных и бизнес-логикой валидации учетных записей.

Модуль управления рецептами содержит всю логику создания, редактирования, поиска и категоризации кулинарных рецептов включая работу с ингредиентами, инструкциями приготовления, изображениями и метриками популярности. Данный модуль имеет слабую связанность с пользовательским модулем только через идентификаторы авторов рецептов что обеспечивает возможность независимого развития и масштабирования.

Каталожный модуль выделен как отдельный сервис справочных данных включающий категории блюд, уровни сложности приготовления и другие классификаторы используемые для структурирования рецептов. Модуль имеет стабильную структуру данных с редкими изменениями что позволяет оптимизировать его для высокой производительности чтения и кэширования справочной информации.

Модуль API Gateway реализует функции маршрутизации запросов, агрегации данных от различных сервисов, централизованной аутентификации и мониторинга производительности системы. Gateway обеспечивает единую точку входа для клиентских приложений скрывая сложность внутренней микросервисной архитектуры и предоставляя консистентный интерфейс для фронтенд разработчиков.

Архитектура системы была реорганизована в четыре независимых микросервиса каждый из которых развертывается в отдельном Docker контейнере с собственной базой данных и конфигурацией. User Service развернут на порту 3001 с PostgreSQL базой kitchen_users и включает endpoints для регистрации пользователей, управления профилями,

аутентификации и получения информации о пользователях по идентификатору и email адресу.

Recipe Service функционирует на порту 3002 с базой данных kitchen_recipes и предоставляет полный CRUD функционал для рецептов включая создание, редактирование, удаление, поиск с фильтрацией по категориям, времени приготовления и ингредиентам. Сервис реализует дополнительные endpoints для получения популярных рецептов, статистики и рецептов конкретного пользователя с поддержкой пагинации и сортировки результатов.

Catalog Service работает на порту 3003 с базой kitchen_catalog и управляет справочными данными включая категории блюд и уровни сложности приготовления. Сервис предоставляет endpoints для получения полного каталога данных, создания новых категорий и уровней сложности, а также статистической информации о количестве элементов в справочниках.

API Gateway развернут на порту 3000 и выполняет функции прокси сервера маршрутизируя запросы к соответствующим микросервисам через http-proxy-middleware. Gateway обрабатывает CORS политики, логирование запросов, health checks всех сервисов и предоставляет агрегированные endpoints для получения статистики и поиска по всем сервисам одновременно.

Межсервисное взаимодействие реализовано через два основных механизма синхронной HTTP коммуникации для запрос-ответ операций и асинхронного обмена сообщениями через RabbitMQ для event-driven архитектуры. HTTP взаимодействие используется для валидации данных между сервисами например Recipe Service проверяет существование пользователя через HTTP запрос к User Service перед созданием нового рецепта.

RabbitMQ message broker развернут в отдельном контейнере и обеспечивает надежную доставку событий между микросервисами через fanout exchange с persistent сообщениями. User Service публикует событие USER_CREATED при регистрации нового пользователя а Recipe Service подписывается на это событие и автоматически создает приветственный рецепт для новых пользователей демонстрируя loose coupling между сервисами.

Сетевая конфигурация Docker Compose создает изолированную bridge сеть kitchen-network где все сервисы могут взаимодействовать друг с другом по именам контейнеров вместо IP адресов что обеспечивает стабильность соединений при перезапуске контейнеров. Каждый сервис имеет health

check для проверки готовности к обслуживанию запросов что позволяет API Gateway корректно обрабатывать недоступность отдельных сервисов.

Load balancing и service discovery реализованы через Docker Compose depends_on конфигурацию которая обеспечивает правильную последовательность запуска сервисов с ожиданием готовности баз данных перед стартом приложений. Environment переменные используются для конфигурации URL сервисов что позволяет легко изменять топологию сети без пересборки Docker образов.

Вывод

Реализованная микросервисная архитектура обеспечивает независимое развертывание и масштабирование сервисов, отказоустойчивость системы при недоступности отдельных компонентов и возможность использования различных технологий в разных сервисах в соответствии с их специфическими требованиями.